

Root Cause Analysis (RCA)

1. Assignment Overview

The purpose of this assignment was to:

- Use Terraform to create Docker resources locally.
 - Create two Docker containers:
 - `web` – Nginx container.
 - `app` – Traefik Whoami container.
 - Make sure both containers can communicate with each other.
 - Make Terraform idempotent.
 - Fix security and best-practice issues.
 - Verify that Terraform can be applied multiple times without unnecessary changes.
-

2. Issues Found

Issue 1: `timestamp()` was causing non-idempotent behavior

- The original Terraform configuration used `timestamp()`.
- `timestamp()` generates a new value when Terraform runs.
- Because the value changes, Terraform can detect changes every time it runs.
- This can cause unnecessary updates or recreation of resources.

Fix

- Removed the use of `timestamp()` from the Terraform configuration.

Result

- Terraform configuration is now stable.
- Running `terraform apply` again should show:

No changes. Your infrastructure matches the configuration.

3. Issue 2: Docker containers needed a common network

- The `web` and `app` containers need to communicate with each other.
- To allow communication, both containers were connected to the same Docker network.

Fix

Created a dedicated Docker network:

```
resource "docker_network" "app_net" {  
  name = "assignment-net"  
}
```

Both containers are connected to:

assignment-net

Result

- The containers can communicate using Docker's internal DNS.
- The `web` container can be accessed using the hostname:

web

4. Issue 3: Container Connectivity

The containers were configured as:

```
app  
|  
| assignment-net  
|  
v  
web (nginx)
```

Fix

- Connected both containers to `assignment-net`.
- Verified that the Docker network contains both containers.
- Verified that the Nginx container is accessible on port `8080`.

5. Security and Best-Practice Improvements

The Terraform configuration was checked for security issues.

Changes

- No passwords or sensitive credentials are hardcoded.
- No API keys or secrets are stored in the Terraform files.
- A dedicated Docker network is used instead of the default Docker network.
- The configuration was reviewed to avoid unnecessary dynamic values.

Docker Images

The configuration currently uses:

```
nginx:latest
traefik/whoami:latest
```

For a production environment, it is better to use a specific image version or image digest instead of `latest`.

For this assignment, the existing image tags were retained because the main focus is Terraform and Docker functionality.

6. Terraform Changes Made

The following changes were made:

- Created the `assignment-net` Docker network.
 - Connected the `web` container to the network.
 - Connected the `app` container to the network.
 - Exposed the Nginx container on port `8080`.
 - Removed the `timestamp()` usage.
 - Checked the configuration for hardcoded passwords and secrets.
 - Added a Terraform output for the web container ID.
-

7. Validation and Testing

The Terraform configuration was tested using the following commands:

```
terraform fmt
terraform init
terraform validate
terraform plan
terraform apply
```

After the first successful `terraform apply`, Terraform was run again:

```
terraform apply
```

The second run should show:

```
No changes. Your infrastructure matches the configuration.
```

Docker resources were checked using:

```
docker ps
```

The Docker network was checked using:

```
docker network inspect assignment-net
```

The Nginx application was tested using:

```
curl http://localhost:8080
```

8. Final Result

After the changes:

- Terraform configuration is valid.
- Docker network is created successfully.
- Both containers are created successfully.
- Both containers are connected to the same Docker network.
- Containers can communicate using Docker networking.
- Nginx is accessible through port `8080`.
- `timestamp()` has been removed.
- No hardcoded passwords or secrets are present.
- Terraform can be run again without unnecessary changes.

9. Conclusion

The main problems were:

- Non-idempotent Terraform configuration because of `timestamp()`.
- Docker networking needed to be configured correctly for container communication.
- Security and best-practice issues needed to be reviewed.

These issues were fixed and the Terraform configuration was tested to make sure the Docker infrastructure works as expected.